Fonology

v 1.1.2

Phonological analysis in R

Guilherme D. Garcia

Pavillon Charles-De Koninck, 1030, Bureau DKN-3267

Département de langues, linguistique et traduction, Université Laval

Avenue des Sciences-Humaines, Québec QC, Canada

gdgarcia.ca guilhermegarcia guilherme.garcia@lli.ulaval.ca

Last updated: 2026-05-05

Questions, suggestions, bugs

Any questions, comments or suggestions should be posted to the repository below (issues). All the bugs you find will help improve the package! Simply open an issue.

- gdgarcia.ca/fonology
- guilhermegarcia/fonology
- guilhermegarcia/fonology/discussions
- guilhermegarcia/fonology/issues

1. Introduction

Back when I was working on my dissertation, I had to create a lexicon for Portuguese with IPA transcription, stress, and other phonological variables. While word lists and corpora were available, they were rarely coded for the variables I needed. Simply put, written data is easy to gather, but difficult to work with if you don't have a good grapheme-phoneme conversion, for example. Since then, I've had to go through a similar set of tasks for different projects, since I'm interested in how the lexicon and the grammar interact. Unsurprisingly, mapping lexical patterns is essential to understand said interaction. In 2022, I started to work on a series of functions that would automate the task for me, and a year later I decided to turn the functions into an R package. Since then, Fonology has had several improvements, and I feel that now it can be useful to others in the field.

In a nutshell, Fonology takes written data and transcribes it using a combination of lexicon lookup and regular expressions. The goal was to create a light and fast package that would be easy to interpret and debug, with transparent phonological mappings that anyone could tweak and customize. This meant avoiding approaches such as neural models. The package currently supports English, French, Italian, Portuguese and Spanish, with different accuracy rates (but all >80%).

While IPA transcription is the most important feature of Fonology, it also offers a wide range of functions to code for phonological variables given a phonemic transcription. The package also has a MaxEnt function for learning weights, different databases, and some helpers. In this manual, I go over all the available functions.

1.1. Installation

To install Fonology, simply run the line below. The package has some dependencies, and is meant to be used alongside Tidyverse (Wickham et al., 2019).

```r
# Option 1:
pak::pak("guilhermegarcia/fonology")

# Option 2:
remotes::install_github("guilhermegarcia/fonology")
```

Code 1: Installing Fonology

2. From graphemes to phonemes

The main function in Fonology is `ipa(word, lg)`, which takes a string of graphemes `word` in language `lg` and returns a phonemic transcription (GRAPHEME-PHONEME CONVERSION, henceforth g2p). For Portuguese inputs, the argument `narrow` (boolean) also offers a narrower transcription, but the user should consider this option as merely illustrative and for teaching purposes only (given that inputs are graphemes, it's not even clear what a narrow transcription would represent in this case).

The transcription will work for both existing and nonce words, but with different levels of accuracy. Because the package relies on lookups and regex, it is therefore completely blind to morphology, because the function isn't meant to tag inputs (and its inputs are expected to carry that information). Combining g2p with tagging would defeat the purpose of the package as it would create a much slower workflow. Naturally, the user can combine Fonology and `udpipe` (Wijffels, 2023) to tag words before transcribing them, but `ipa()` will not use this information for g2p.

For all supported languages except English, regex can be surprisingly accurate. Italian, Portuguese and Spanish have a fairly predictable g2p. French is trickier, but increasing the grapheme window already helps bring accuracy up to >80%. To get even more accurate results, Portuguese, French, and English now follow a lookup-first workflow. Portuguese lookup is based on the Portuguese Stress Lexicon (Garcia, 2014). French user IPA overrides in `fr_ipa_lex` take priority, then Fonology checks the Lexique 4-derived `fr_lex` data (New et al., 2026), and only then falls back to regular-expression rules for out-of-vocabulary forms. Regex-derived Portuguese, French, and English forms are marked with a final `*`; helper functions ignore this marker when parsing phonological material. In a test on *Madame Bovary*, roughly 90% of all word tokens were covered by the lookup path and therefore required no regex fallback; after removing stopwords, lookup coverage was roughly 80%. This makes the starred forms a compact audit list for manual checking or lexicon expansion. For all languages, a custom lexicon is also used for reference. This custom lexicon is basically a dataset of unpredictable forms, and can be expanded as needed with specific functions.

How about English? The language is famously complicated when it comes to g2p. For that reason, Fonology uses two lookups before defaulting to regex. The first pass uses the CMU Dictionary (CMUdict, 2014). The second pass includes a complementary lexicon (just like the other languages mentioned above). Only then do regex rules apply. If an output was generated solely with regex (nonce or rare words), you will see an asterisk appended at the end of each word. This is an easy way to keep track of all words that *might* be problematic in a corpus. For reference, only about 3% of all types in *Moby Dick* relied on regex alone. Everything else was caught in the lookups — which doesn't necessarily mean that 97% of all words in the novel are perfectly coded for, given that tagging wasn't part of the analysis. Therefore, while English is indeed the most challenging case here, it is also the most reliable because CMU is such a great resource. Portuguese is the oldest language in the package, and PSL-backed lookup now makes the regex-only path explicit there too. To improve the remaining regex fallback

cases for French, Italian and Spanish, people need to use the package, find problems, and report them in the repository.

```
"I like coffee and poggy" |>
  cleanText() |>
  ipa(lg = "en")
[1] "'aɪ"     "'laɪk"   "'kɑ.fi"  "ənd"     "'pɑ.gɪ*"
```

Code 2: Code

Code 2 is a simple example that also introduces another function, `cleanText()`. The role of this function is to, well, clean the string before `ipa()` works on it. The output here has one word it couldn't find in its lookups, *poggy*, hence the asterisk at the end of the word. You can later fix/remove all words ending in an asterisk to make sure you only rely on lookups, for example. Code 3 shows `ipa()` for the other supported languages in the package. You can run `ipa_xx_test()` to print a sample of test words for all supported languages (just replace `xx` with the language).

```
ipa("répété", lg = "fr")
[1] "ʁe.pe.te"
ipa("buonasera", lg = "it")
[1] "bwo.na.'se.ra"
ipa("coração", lg = "pt")
[1] "ko.ra.'sãw̃"
ipa("tiene", lg = "sp")
[1] "'tje.ne"
```

Code 3: One-word inputs for French, Italian, Portuguese and Spanish

Given the average accuracy mentioned earlier, it goes without saying that there will be errors and mismatches, especially when it comes to stress marks. Sometimes, stress will be incorrect because of morphology, i.e., there's nothing the function could have done. But sometimes the error stems from a faulty generalization. Both cases are easy to adjust (either with better regex or with a better database for the lookups). Once we have g2p, everything else is much easier to accomplish. In the next section, other functions are discussed that help extract a wide range of phonological variables, from syllabic constituents to stress.

3. Helper functions

The functions listed below are meant to be used *after* `ipa()` has been applied, so their input is assumed to be phonemically transcribed already.

Function	What it does
`countSyl()`	Counts syllables in an already syllabified transcription.
`cv()`	Returns the C/V/G syllable-shape profile of an `ipa()` output.
`getSyl()`	Extracts a specific syllable from a syllabified transcription.
`getStress()`	Labels stress position or returns the stressed syllable.
`getWeight()`	Returns the syllable-weight profile of a transcribed word as `L` / `H`.
`syllable()`	Extracts the onset, nucleus, coda, or rhyme of a syllable.

Perhaps the best way to visualize these function in action is through an example. Code 4 downloads the complete text for *Moby Dick* from Project Gutenberg using the `gutenbergr` package (Bradford et al., 2026). Both `tidyverse` (Wickham et al., 2019) and `tidytext` (Silge & Robinson, 2017) are used here as well. Examine the variable `md`, which is created from the raw text in question. After tokenizing the text using the `unnest_tokens()` function, we remove function words — the object `stopwords_en` comes with Fonology and is derived from the `stopwords` package (Benoit et al., 2021).

The crucial step in Code 4 is the creation of several columns that code for phonological variables: IPA transcription, CV shape, weight profile, and stress. We also extract the final syllable with the `getSyl()` function and, finally, we add a column for the onset of the final syllable using the `syllable()` function. The functions used here complement `ipa()`, and all of them rely on g2p in the data. The creation of `md` takes about 15s, but could be sped up if we focused on unique words, for example. The resulting tibble is shown in Output 1.

```r
library(gutenbergr)
library(Fonology)
library(tidyverse)
library(tidytext)

gutenberg_metadata |>
  filter(author == "Melville, Herman") |>
  select(gutenberg_id, title) |>
  print(n = 20)

moby_raw <- gutenberg_download(2701) |> select(text)

md <- moby_raw |>
  unnest_tokens(word, text) |>
  filter(!word %in% stopwords_en) |>
  mutate(
    ipa = ipa(word, lg = "en"),
    cv = cv(ipa),
    weight = getWeight(ipa, lg = "en"),
    stress = getStress(ipa),
    finSyl = getSyl(ipa, 1),
    onsetFin = syllable(finSyl, const = "onset", glides_as_onsets = TRUE)
  )
```

Code 4: Downloading *Moby Dick* and extracting phonological variables from text

```
# A tibble: 81,958 × 7
   word      ipa            cv            weight stress     finSyl onsetFin
   <chr>     <chr>          <chr>         <chr>  <chr>      <chr>  <chr>
 1 moby      'moʊ.bi         CVV.CV        HL     penult     bi     b
 2 dick      'dɪk            CVC           H      final      dɪk    d
 3 whale     'weɪl           GVVC          H      final      weɪl   w
 4 herman    'hɝ.mən         CV.CVC        LH     penult     mən    mən
 5 melville  'mɛl.vɪl        CVC.CVC       HH     penult     vɪl    v
```

Output 1: Resulting tibble for *Moby Dick*

The workflow illustrated above works for all the supported languages. The user can examine the available functions in the documentation to better understand the necessary and optional arguments.

4. Distinctive features

Two functions in Fonology are dedicated to distinctive features. First, `getFeat()` takes a set of phonemes `ph` in language `lg` and returns a minimal matrix. Second, `getPhon()` takes a set of features `ft` in language `lg` and returns the phonemes described by the matrix in question. While both functions rely on some preset languages, they also allow the user to set their own inventory. Examples are shown in Code 5.

```
# From phonemes to features
getFeat(ph = c("i", "u"), lg = "English")
#> [1] "+hi"    "+tense"
getFeat(ph = c("i", "u"), lg = "French")
#> [1] "Not a natural class in this language."
getFeat(ph = c("i", "y", "u"), lg = "French")
#> [1] "+syl" "+hi"
getFeat(ph = c("p", "b"), lg = "Portuguese")
#> [1] "-son"  "-cont" "+lab"
getFeat(ph = c("k", "g"), lg = "Italian")
#> [1] "+cons" "+back"

# From features to phonemes
getPhon(ft = c("+syl", "+hi"), lg = "French")
#> [1] "u" "i" "y"
getPhon(ft = c("-DR", "-cont", "-son"), lg = "English")
#> [1] "t" "d" "b" "k" "g" "p"
getPhon(ft = c("-son", "+vce"), lg = "Spanish")
#> [1] "z" "d" "b" "ʝ" "g" "v"

# Your own inventory:
getFeat(ph = c("p", "f", "w"),
  lg = c("a", "i", "u", "y", "p",
         "t", "k", "s", "w", "f"))
#> [1] "-syl" "+lab"

getPhon(ft = c("-son", "+cont"),
        lg = c("a", "i", "u", "s", "z",
         "f", "v", "p", "t", "m"))
#> [1] "s" "z" "f" "v"
```

Code 5: Distinctive features in Fonology

5. Maxent

The function `maxent()` estimates weights in a Maximum Entropy Grammar (Goldwater & Johnson, 2003; Hayes & Wilson, 2008) given a tableau object containing inputs, outputs, constraints, violations and observations (see documentation). The function returns a list with different objects, including learned weights, BIC value and predicted probabilities for each output. The user will also find functions for noisy harmonic grammars in the package. An illustrative tableau is provided (`maxent_tableau_1`), so the user can easily see what the function

looks like simply by running `maxent_tableau_1 ▷ maxent()`. Code 6 demonstrates how a tableau is created before running `maxent()`. Finally, I strongly recommend the `maxent.ot` package (Mayer et al., 2024).

```
maxent_data <- tibble::tibble(
  input = rep(c("pad", "tab", "bid", "dog", "pok"), each = 2),
  output = c("pad", "pat", "tab", "tap", "bid", "bit", "dog", "dok", "pog", "pok"),
  ident_vce = c(0, 1, 0, 1, 0, 1, 0, 1, 1, 0),
  no_vce_final = c(1, 0, 1, 0, 1, 0, 1, 0, 1, 0),
  obs = c(5, 15, 10, 20, 12, 18, 12, 17, 4, 8)
)
maxent(tableau = maxent_data)
#> $predictions
#> # A tibble: 10 × 12
#>    input output ident_vce no_vce_final   obs harmony max_h exp_h     Z obs_prob
#>    <chr> <chr>      <dbl>        <dbl> <dbl>   <dbl> <dbl> <dbl> <dbl>    <dbl>
#>  1 pad   pad            0            1     5  0.639  0.639  1     2.79    0.25
#>  2 pad   pat            1            0    15  0.0541 0.639  1.79  2.79    0.75
#>  3 tab   tab            0            1    10  0.639  0.639  1     2.79    0.333
#>  4 tab   tap            1            0    20  0.0541 0.639  1.79  2.79    0.667
#>  5 bid   bid            0            1    12  0.639  0.639  1     2.79    0.4
#>  6 bid   bit            1            0    18  0.0541 0.639  1.79  2.79    0.6
#>  7 dog   dog            0            1    12  0.639  0.639  1     2.79    0.414
#>  8 dog   dok            1            0    17  0.0541 0.639  1.79  2.79    0.586
#>  9 pok   pog            1            1     4  0.693  0.693  1     3.00    0.333
#> 10 pok   pok            0            0     8  0      0.693  2.00  3.00    0.667
#> # ℹ 2 more variables: pred_prob <dbl>, error <dbl>
#>
#> $weights
#>    ident_vce no_vce_final
#>   0.05410679   0.63904039
#>
#> $log_likelihood
#> [1] -78.72152
#>
#> $log_likelihood_norm
#> [1] -7.872152
#>
#> $bic
#> [1] -152.8379
```

Code 6: A MaxEnt grammar in Fonology

Given that the object generated by `maxent()` is a list, it is easy to extract different components of the analysis. For example, if we assign `maxent()` to a variable `my_maxent`, then we'd extract weights by running `my_maxent$weights`. For a complete but concise demo comparing English and Portuguese phonotactics from two novels, see this blog post.

6. Additional functions

While the main functions available in Fonology are those already discussed, some extra functions also come in the package. Below, I briefly go over some of these functions.

6.1. Sonority

There are three functions in the package to analyze sonority. First, `demi(word, d)` extracts either the first (`d = 1`, the default) or second (`d = 2`) demisyllable of a given (syllabified) word (or vector of words). Second, `sonDisp(demi)` calculates the sonority dispersion score of a given demisyllable (Clements, 1990). Note that this metric does not differentiate sequences that respect the sonority sequencing principle (SSP) from those that do not, i.e., `pla` and `lpa` will have the same score. For that reason, a third function exists, `ssp(demi, d)`, which evaluates whether a given demisyllable respects (`1`) or doesn't respect (`0`) the SSP.

An extra function dedicated to plotting sonority profiles is also available, `plotSon()`. The user can run, for example, `"combradol" ▷ ipa() ▷ plotSon()`. This will generate a `ggplot2` object showing the sonority profile of the nonce word in question, assuming Portuguese (default language in `ipa()`). This type of function can be useful in introductory courses, although it has been superseded by `phonokit` (Garcia, 2026) if you use Typst.

6.2. Bigrams probabilities for Portuguese

The function `biGram_pt()` returns the log bigram probability for a possible `word` in Portuguese (where `word` is phonemically transcribed using `ipa()`) — but note that the string must not contain syllable boundaries nor stress marks. The easiest way to use the function in question is to pipe it into `ipa()`, as usual: `"paclode" ▷ ipa() ▷ biGram_pt()`. The calculation is based on the Portuguese Stress Lexicon (Garcia, 2014), so the probabilities are derived from the actual lexicon.

6.3. Word generator for Portuguese

The function `wug_pt()` generates a hypothetical word in Portuguese. Note that this function is meant to be used to get you started with nonce words. You will most likely want to make adjustments based on phonotactic preferences. The function already takes care of some OCP effects and it also prohibits more than one onset cluster per word, since that's relatively rare in Portuguese. Still, there will certainly be other sequences that sound less natural. The function is not too strict because you may have a wide range of variables in mind as you create novel words. Finally, if you wish to include palatalization, set `palatalization = T` — if you do that, `biGram_pt()` will de-palatalize words for its calculation, as it's based on phonemic transcription.

6.4. Plotting vowels

The function `plotVowels()` creates a vowel trapezoid using `ggplot2`. If `tex = T`, the function also saves a `tex` file with the LaTeX code to create the same trapezoid using the `vowel` package. The available languages are Arabic, French, English, Dutch, German, Hindi, Italian, Japanese, Korean, Mandarin, Portuguese, Spanish, Swahili, Russian, Italian, Thai, and Vietnamese. Only oral monophthongs are plotted. This function is also implemented as a Shiny App here. However, if you use Typst, this function has also been superseded by `phonokit` (Garcia, 2026).

6.5. Ages in acquisition studies

`monthsAge()` is a minor function that converts `aa;bb` to age in months, which then allows for quick calculations such as mean age for any data set.

```r
monthsAge(age = "02;06")
#> [1] 30
monthsAge(age = "05:03", sep = ":")
#> [1] 63
meanAge(age = c("02;06", "03;04", NA))
#> [1] "2;11"
meanAge(age = c("05:03", "04:07"), sep = ":")
#> [1] "4:11"
```

Code 7: Working with ages

6.6. Typesetting transcriptions

When I used LaTeX, I used the excellent `tipa` package (Rei, 1996). The function `ipa2tipa()` takes a transcribed input and returns the `tipa` code to reproduce the string in LaTeX. As long as `ipa()` is applied first, the function will work over single inputs or vectors. In Code 8, `cleanText()` is used to prepare the input for `ipa()` — this has a similar effect as `tidytext`'s `unnest_tokens()` used in Code 4. Finally, if you use Typst, `ipa2typst()` does exactly the same thing, but assuming the `phonokit` package instead of `tipa`.

```r
"This is a common sentence in English" |> cleanText() |>  ipa(lg = "en") |> ipa2tipa()
#> Done! Here's your tex code using TIPA:
#> \textipa{ / "DIs "Iz @ "kA.m@n "sEn.t@ns In "IN.glIS / }

"This is a common sentence in English" |> cleanText() |>  ipa(lg = "en") |> ipa2typst()
#> Done! Here's your Typst code using phonokit:
#> #ipa("'DIs \\s 'Iz \\s @ \\s 'kA.m@n \\s 'sEn.t@ns \\s In \\s 'IN.glIS")
```

Code 8: Typesetting transcriptions

References

Benoit, K., Muhr, D., & Watanabe, K. (2021). *stopwords: Multilingual Stopword Lists*. https://doi.org/10.32614/CRAN.package.stopwords

Bradford, J., Harmon, J., Johnston, M., & Robinson, D. (2026). *gutenbergr: Download and Process Public Domain Works from Project Gutenberg*. https://doi.org/10.32614/CRAN.package.gutenbergr

Clements, G. N. (1990). The role of the sonority cycle in core syllabification. In J. Kingston & M. Beckman (Eds.), *Papers in Laboratory phonology 1: Between the grammar and physics of speech: Papers in Laboratory phonology 1: Between the grammar and physics of speech* (pp. 283–333). Cambridge University Press.

CMUdict. (2014). *The Carnegie Mellon pronouncing dictionary*. http://www.speech.cs.cmu.edu/cgi-bin/cmudict

Garcia, G. D. (2014). *Portuguese stress lexicon*.

Garcia, G. D. (2026,). *phonokit: a toolkit to create phonological representations in Typst*. Zenodo. https://doi.org/10.5281/zenodo.18434478

Goldwater, S., & Johnson, M. (2003). Learning OT constraint rankings using a Maximum Entropy model. *Proceedings of the Stockholm Workshop on Variation within Optimality Theory*, 111–120.

Hayes, B., & Wilson, C. (2008). A Maximum Entropy Model of Phonotactics and Phonotactic Learning. *Linguistic Inquiry*, *39*(3), 379–440. https://doi.org/10.1162/ling.2008.39.3.379

Mayer, C., Tan, A., & Zuraw, K. R. (2024). Introducing \texttt{maxent.ot}: An R package for Maximum E ntropy constraint grammars. *Phonological Data and Analysis*, *6*(4), 1–44. https://doi.org/10.3765/pda.v6art4.88

New, B., Pallier, C., Schalchli, G., Bourgin, J., & Gimenes, M. (2026). Lexique 4: A major upgrade of the Lexique French lexical database. *Behavior Research Methods*.

Rei, F. (1996). TIPA: A system for processing phonetic symbols in LATEX. *TUGBoat*.

Silge, J., & Robinson, D. (2017). *Text mining with R: A tidy approach*. O'Reilly Media, Inc.

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., … Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. https://doi.org/10.21105/joss.01686

Wijffels, J. (2023). *udpipe: Tokenization, Parts of Speech Tagging, Lemmatization and Dependency Parsing with the 'UDPipe' 'NLP' Toolkit*. https://cran.r-project.org/package%20=%20udpipe